

專題二-飛機訂票

班級：E E I T 9 4

組別：第五組

組長：24 蔡岳峰

副組長：08 劉孟儒

成員：07 郭俊宏、

14 蔣方怡



主題說明

本系統是一個面向B2C的飛航線上訂票平台，提供航班查詢、座位選擇、訂單管理、用戶管理、里程兌換等功能。

透過Java、SQL Server、Servlet等技術，確保系統安全、高效且易於使用。



系統功能與分工



劉孟儒

用戶系統



蔡岳峰

票務系統



郭俊宏

航班系統

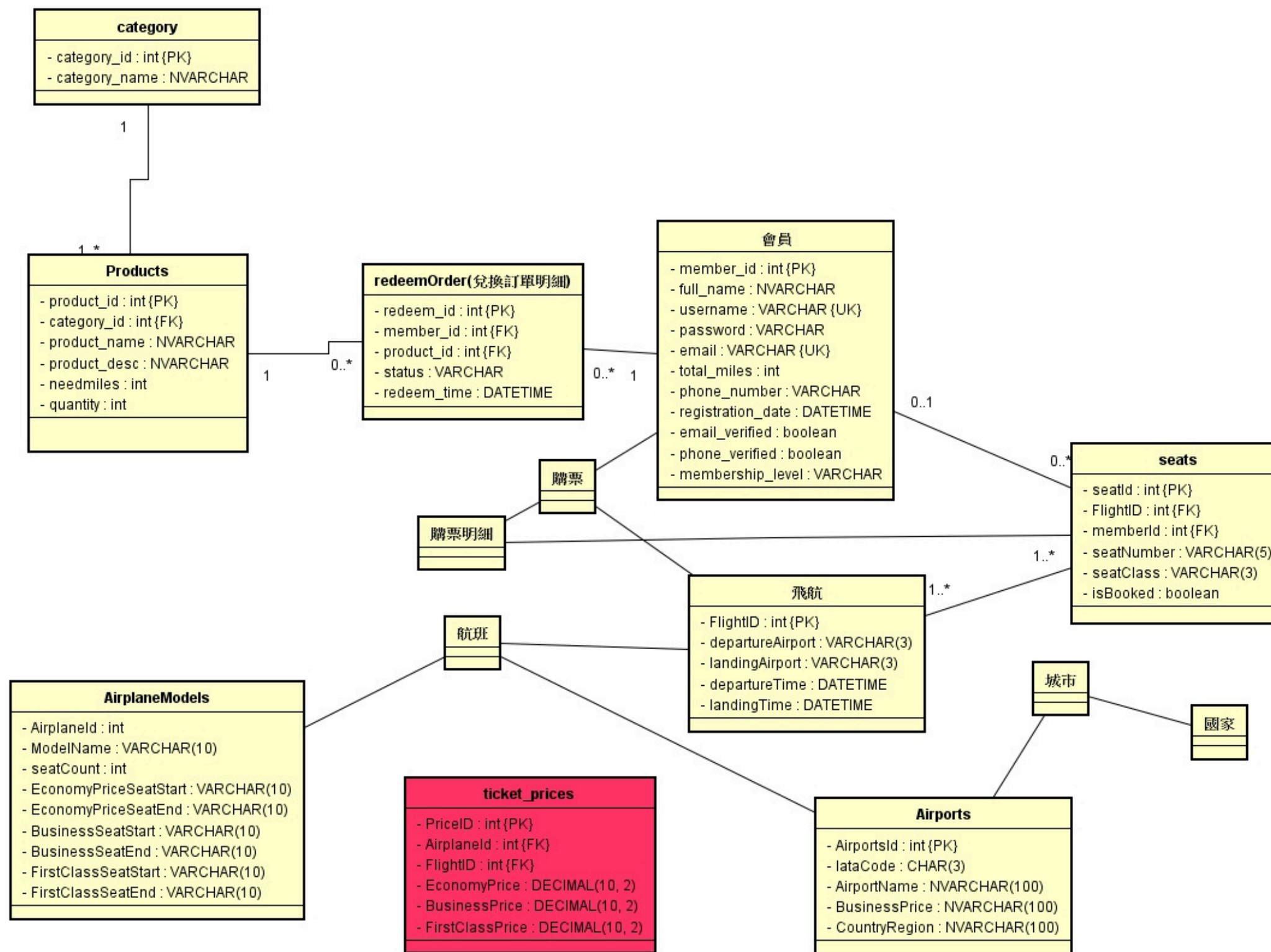


蔣方怡

里程兌換

1. 08_劉孟儒:用戶(客戶、管理員、註冊、登入)
2. 24_蔡岳峰:票務(訂票狀態、訂票、選位、訂單、機場)
3. 07_郭俊宏:航班(航線、機型、搜尋航班)
4. 14_蔣方怡:里程兌換(商品管理、訂單管理、商品兌換)

ER Model



頁面版型

CSS

用css統一頁面側邊欄版型，以及畫面設計
在html檔用link rel載入 stylesheet

```
body {  
    font-family: Arial, sans-serif;  
    margin: 0;  
    padding: 0;  
    background-color: #f4f4f9;  
}  
  
.sidebar {  
    height: 100%;  
    width: 250px;  
    position: fixed;  
    top: 0;  
    left: 0;  
    background-color: #333;  
    padding-top: 20px;  
}  
  
.sidebar a {  
    padding: 15px 20px;  
    text-decoration: none;  
    font-size: 18px;  
    color: white;  
    display: block;  
}  
  
.sidebar a:hover {  
    background-color: #575757;  
}  
  
.content {  
    margin-left: 260px;  
    /* 留出側邊欄的空間 */  
}
```

動態載入導航欄

使用javascript定義一個函數
loadNav，用來加載導航欄

使用 fetch API 請求 navtest.html
文件

將獲取的數據插入到 id 為 'nav' 的元
素中

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8"> <!-- 設定字符編碼為 UTF-8 -->
5   <meta name="viewport" content="width=device-width, initial-scale=1.0"> <!-- 設定視口以
6   <title>Document</title> <!-- 頁面標題 -->
7 </head>
8 <script>
9   // 定義一個函數 loadNav，用來加載導航欄
10  function loadNav() {
11    // 使用 fetch API 請求 navtest.html 文件
12    fetch('navtest.html')
13      .then(response => response.text()) // 將響應轉換為文本
14      .then(data => {
15        // 將獲取的數據插入到 id 為 'nav' 的元素中
16        document.getElementById('nav').innerHTML = data;
17      });
18  }
19 </script>
20 <body onload="loadNav()"> <!-- 當頁面加載完成時，自動調用 loadNav 函數 -->
21   <div id="nav"></div> <!-- 用於顯示導航欄的容器 -->
22 </body>
23 </html>
```


會員管理

航線管理

機場管理

贈品管理

登出

用戶系統

1. 管理員後台登入系統

- 登入 - 登出 - 驗證 - 註冊

2. 會員管理

- 新增 - 修改 - 刪除 - 查詢



管理員登入系統

登入畫面

1. 以現有的帳號密碼登入
2. 或註冊一個新的管理員帳密

驗證

1. 確認帳號是否存在
2. 對應密碼是否正確
3. 獲取Session並導入首頁

登出

1. 透過左側導覽列登出系統
2. 中止Session使其失效

Admin

表格名稱	中文：管理員資料庫 英文：Admin		索引鍵：		
主鍵：	admin_id		外來鍵：		
欄位編號	欄位名稱	欄位敘述	資料型態	欄位長度	備註
1	admin_id	管理員ID	INT		PK
2	full_name	管理員姓名	NVARCHAR	100	
3	username	管理員帳號名稱	VARCHAR	50	UK
4	password	管理員密碼	VARCHAR	255	
5	email	管理員電子郵件	VARCHAR	100	UK
6	phone_number	管理員電話號碼	VARCHAR	20	
7	registration_date	註冊時間	DATETIME		

註冊

管理員登入

使用者名稱

密碼

登入

[註冊新帳號](#)



註冊管理員 HTML

新增對應資料傳至後端處理

在此簡單驗證email、
電話及註冊時間的格式

```
<h2>新增管理員</h2>

<form method="post" action="../../NewAdmin">
  <div class="form-group">
    <label for="full_name">新增管理員姓名:</label>
    <input type="text" id="full_name" name="full_name" required />
  </div>

  <div class="form-group">
    <label for="username">新增使用者名稱:</label>
    <input type="text" id="username" name="username" required />
  </div>

  <div class="form-group">
    <label for="password">新增管理員密碼:</label>
    <input type="password" id="password" name="password" required />
  </div>

  <div class="form-group">
    <label for="email">新增管理員電子郵件:</label>
    <input type="email" id="email" name="email" required />
  </div>

  <div class="form-group">
    <label for="phone_number">新增管理員電話號碼:</label>
    <input type="text" id="phone_number" name="phone_number" required pattern="^09\d{8}$" title="電話號碼必須以09開頭並且為10位數字" />
  </div>

  <div class="form-group">
    <label for="registration_date">新增管理員註冊時間:</label>
    <input type="date" id="registration_date" name="registration_date" required />
  </div>

  <input type="submit" value="確定" />
</form>
```

註冊管理員 Controller

將輸入值轉換成所需的格式

透過Dao內insert方法新增至
admin資料表

向下傳遞一個 AdminBean

```
protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {

    String full_name = request.getParameter("full_name");
    String username = request.getParameter("username");
    String password = request.getParameter("password");
    String email = request.getParameter("email");
    String phone_number = request.getParameter("phone_number");

    //處理日期類型
    String registration_date_str = request.getParameter("registration_date");
    SimpleDateFormat dateFormat = new SimpleDateFormat("yyyy-MM-dd");
    java.util.Date parsedDate = null;
    try {
        parsedDate = dateFormat.parse(registration_date_str);
    } catch (ParseException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }
    java.sql.Date registration_date = new java.sql.Date(parsedDate.getTime());

    AdminDAO admin = new AdminDAO();

    AdminBean admin_ans = admin.insert(full_name,username,password,email,phone_number,registration_date);
    request.setAttribute("admin", admin_ans);
    request.getRequestDispatcher("/topic2/admin/NewAdmin.jsp").forward(request, response);
}
```

連線池

Model

使用DataSource取得
資料庫連線

```
public AdminBean insert(String full_name,String username,String password,String email,  
  
    AdminBean admin = new AdminBean();  
  
    Context context;  
    try {  
        context = new InitialContext();  
        DataSource ds = (DataSource)context.lookup("java:/comp/env/jdbc/servvdb");  
        conn = ds.getConnection();  
        PreparedStatement stmt = conn.prepareStatement(SQL_insert);  
//        stmt.setInt(1, member_id);  
        stmt.setString(1, full_name);  
        stmt.setString(2, username);  
        stmt.setString(3, password);  
        stmt.setString(4, email);  
        stmt.setString(5, phone_number);  
        stmt.setDate(6, registration_date);  
  
        stmt.execute();
```


註冊管理員

View

jsp:useBean取得request內資料

將註冊成功的資料顯示在螢幕上

超連結導回登入頁面

```
6 <meta charset="UTF-8">
7 <title>管理員資料</title>
8 </head>
9 <body style="background-color:#fdf5e6">
10 <div align="center">
11 <h2>新增的管理員資料</h2>
12 <jsp:useBean id="admin" scope="request" class="com.ispan.bean.AdminBean" />
13 <table>
14
15
16 <tr><td>管理員姓名<td><input type="text" disabled value="<%=admin.getFull_name() %>">
17 <tr><td>使用者名稱<td><input type="text" disabled value="<%=admin.getUsername() %>">
18 <tr><td>管理員密碼<td><input type="text" disabled value="<%=admin.getPassword() %>">
19 <tr><td>管理員電子郵件<td><input type="text" disabled value="<%=admin.getEmail() %>">
20
21 <tr><td>管理員電話號碼<td><input type="text" disabled value="<%=admin.getPhone_number() %>">
22 <tr><td>註冊時間<td><input type="text" disabled value="<%=admin.getRegistration_date() %>">
23
24
25
26
27 </table>
28 <a href="http://localhost:8080/FlightTicketingSystem/topic2/login.html">返回登入頁面</a>
29 </div>
30
```


登入

管理員登入

登入

[註冊新帳號](#)



管理員登入 Controller

接收使用者輸入的帳密

執行登入行動前，確認

1.帳號是否存在 2.對應密碼是否正確

正確：給Session

錯誤：跳轉至錯誤訊息頁面

```
public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain) throws IOException, Se
```

```
System.out.println("Authentication filter triggered");
```

```
String username = request.getParameter("username");  
String password = request.getParameter("password");  
AdminDAO admin = new AdminDAO();  
String password_search = admin.getPassword(username);  
System.out.println("搜尋到的:"+password_search);
```

```
if (password_search == null) {  
    // 帳號不存在，直接跳轉到錯誤頁面  
    System.out.println("查無此帳號");  
    HttpServletResponse httpResponse = (HttpServletResponse) response;  
    httpResponse.sendRedirect("topic2/errorPage.jsp?error=username not found");  
    return; // 這裡應該結束執行，避免繼續執行後面的邏輯  
}
```

```
// 如果帳號存在，檢查密碼  
if (password.equals(password_search)) {  
    System.out.println("密碼正確");  
    HttpServletRequest httpRequest = (HttpServletRequest) request;  
    HttpSession session = httpRequest.getSession(); //密碼正確的話 給session  
    session.setAttribute("username", username);
```

```
    chain.doFilter(request, response); // 密碼正確，繼續處理  
} else {  
    System.out.println("密碼錯誤");  
    HttpServletResponse httpResponse_password = (HttpServletResponse) response;
```


密碼錯誤

發生錯誤

Incorrect password

[返回登入頁面](#)

```
-----  
</head>  
<body>  
  <div class="container">  
    <h2>發生錯誤</h2>  
    <p>  
      <%= request.getParameter("error") %>  
    </p>  
    <a href="Login.html">返回登入頁面</a>  
  </div>  
</body>  
</html>
```



會員系統

會員管理

航線管理

機場管理

贈品管理

登出

會員資料處理

取得全部會員資料

輸入會員姓名:

確定

新增會員資料

Member

表格名稱	中文：會員資料庫 英文：Member		索引鍵：		
主鍵：	member_id		外來鍵：		
欄位編號	欄位名稱	欄位敘述	資料型態	欄位長度	備註
1	member_id	會員ID	INT		PK
2	full_name	會員姓名	NVARCHAR	100	
3	username	使用者名稱	VARCHAR	50	UK
4	password	加密後的會員密碼	VARCHAR	255	
5	email	會員電子郵件	VARCHAR	255	UK
6	total_miles	可用里程	INT		
7	phone_number	會員電話號碼	VARCHAR	20	
8	registration_date	註冊時間	DATETIME		
9	email_verified	郵件是否驗證	BOOLEAN		
10	phone_verified	電話是否驗證	BOOLEAN		
11	membership_level	會員等級	VARCHAR	20	

新增會員 HTML

新增對應資料傳至後端處理

格式:

email: email

registration_date: date

membership_level: select, option

email_verified: checkbox

```
<label for="email">新增會員電子郵件:</label>
<input type="email" id="email" name="email" required />
</div>

<div class="form-group">
  <label for="total_miles">新增可用里程:</label>
  <input type="text" id="total_miles" name="total_miles" required />
</div>

<div class="form-group">
  <label for="phone_number">新增會員電話號碼:</label>
  <input type="text" id="phone_number" name="phone_number" required pattern="^\d{8}$" title=
</div>

<div class="form-group">
  <label for="registration_date">新增會員註冊時間:</label>
  <input type="date" id="registration_date" name="registration_date" required />
</div>

<div class="form-group">
  <label for="membership_level">新增會員等級:</label>
  <select id="membership_level" name="membership_level" required>
    <option value="1">1</option>
    <option value="2">2</option>
    <option value="3">3</option>
  </select>
</div>
```

新增會員 Controller

將輸入值轉換成所需的格式

透過Dao內insert方法新增至
admin資料表

向下傳遞一個 MemberBean

此處會將密碼加密

```
String full_name = request.getParameter("full_name");  
String username = request.getParameter("username");
```

```
String password = request.getParameter("password");
```

```
String password_Hashing = PasswordHashing.hashPassword(password);  
System.out.println(password_Hashing);
```

```
String email = request.getParameter("email");  
int total_miles = Integer.parseInt(request.getParameter("total_miles"));  
String phone_number = request.getParameter("phone_number");
```

//處理日期類型

```
String registration_date_str = request.getParameter("registration_date");  
SimpleDateFormat dateFormat = new SimpleDateFormat("yyyy-MM-dd");  
java.util.Date parsedDate = null;  
try {  
    parsedDate = dateFormat.parse(registration_date_str);  
} catch (ParseException e) {  
    // TODO Auto-generated catch block  
    e.printStackTrace();  
}  
java.sql.Date registration_date = new java.sql.Date(parsedDate.getTime());
```

```
boolean email_verified = request.getParameter("email_verified") != null;  
boolean phone_verified = request.getParameter("phone_verified") != null;  
String membership_level = request.getParameter("membership_level");
```

```
MemberDAO member = new MemberDAO();
```

```
MemberBean member_ans = member.insert(full_name,username,password_Hashing,email,total_m  
request.setAttribute("member", member_ans);  
request.getRequestDispatcher("/topic2/InsertMember.jsp").forward(request, response);
```

密碼加密

透過演算法將密碼加密

並將加密結果匯入資料表內

透過資料表無法得知會員密碼

```
public class PasswordHashing {  
    public static void main(String[] args) {  
        String password = "mySecretPassword"; // 你的原始密碼  
        String hashedPassword = PasswordHashing.hash(password);  
        // 將加密後的密碼存入資料庫  
        db.insert("users", hashedPassword);  
    }  
}
```

新增的會員資料

會員姓名	阿瑋
使用者名稱	123
加密後的會員密碼	17e944035afda99377ab424l
會員電子郵件	demkfe@gmail.com
可用里程	12300
會員電話號碼	0966332222
註冊時間	2025-03-20
郵件是否驗證	true
電話是否驗證	false
會員等級	1

[返回](#)

```
} {  
    // 使用 SHA-256 演算法  
    MessageDigest md = MessageDigest.getInstance("SHA-256");  
    byte[] passwordBytes = password.getBytes();  
    md.update(passwordBytes);  
    byte[] hashedPassword = md.digest();  
    return hashedPassword;  
}
```




里程兌換系統

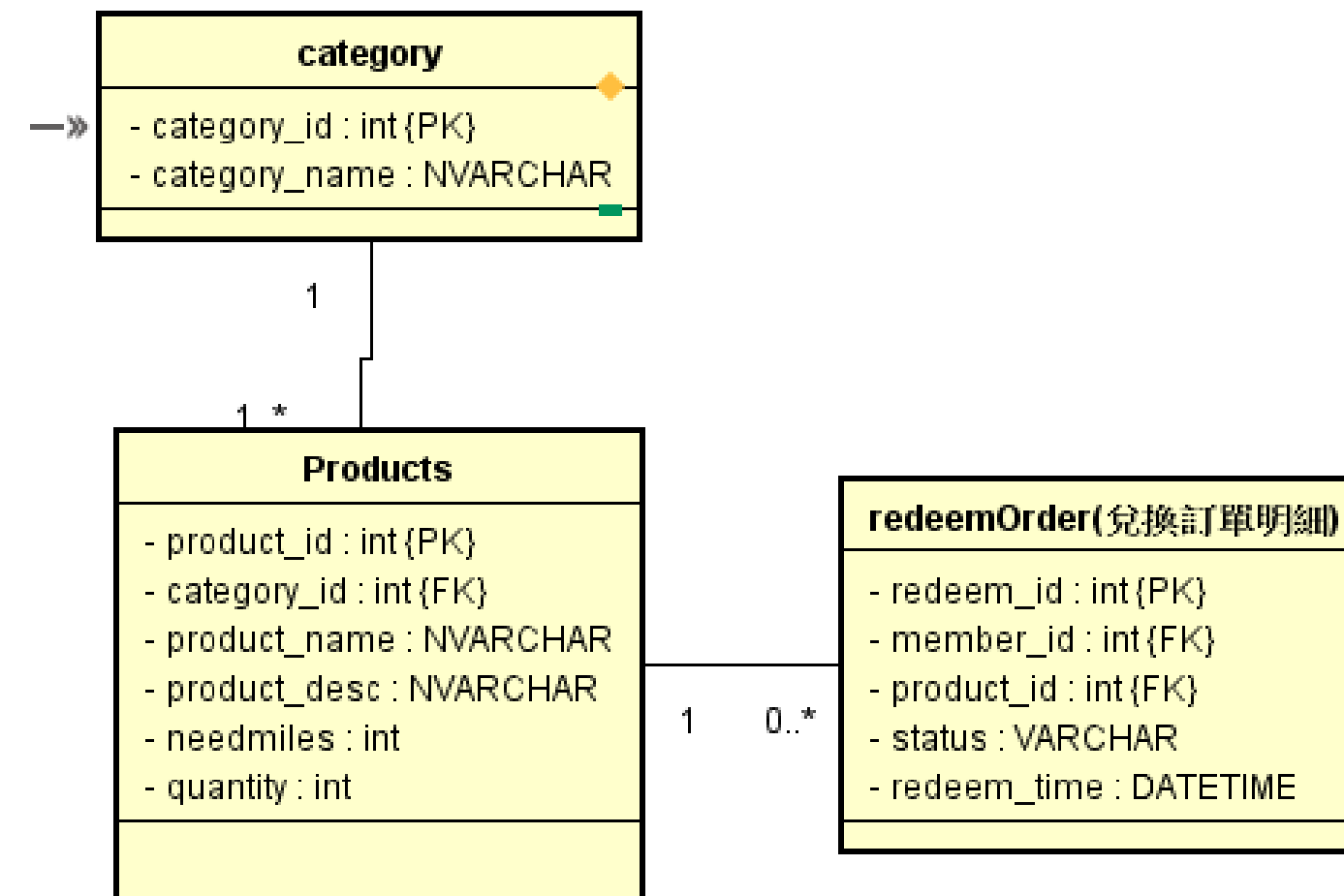
商品後臺管理系統：

- 商品新增
- 商品修改
- 商品刪除
- 商品查詢

負責組員： 蔣方怡



ER Model



SQL指令

(商品類別、商品)

--商品類別表(類別id手動設置)

```
CREATE TABLE Category (  
    category_id INT PRIMARY KEY,  
    category_name NVARCHAR(100) NOT NULL  
);
```

--商品表

```
CREATE TABLE Products (  
    product_id INT IDENTITY(1,1) PRIMARY KEY,  
    category_id INT ,  
    product_name NVARCHAR(100) NOT NULL,  
    product_desc NVARCHAR(400),  
    needmiles INT NOT NULL,  
    quantity INT NOT NULL,  
    product_image NVARCHAR(250),  
    FOREIGN KEY (category_id) REFERENCES Category (category_id) ON DELETE SET NULL  
);
```

里程兌換系統

里程兌換系統的MVC

model

ProductsBean.java

view

Fail.jsp
GetAllProducts.jsp
GetUpdate.jsp
InsertProduct.jsp
RedeemHomePage.jsp
style.css

controller

InsertProduct.java
DeleteProduct.java
GetUpdate.java
UpdateById.java
GetAllProducts.java
GetByName.java
ProductsBean.java

Dao

RedeemDao.java

Utils

utils.java

Product Model

```
package hw1;

public class ProductsBean implements java.io.Serializable {

    private static final long serialVersionUID = 1L;

    //物件屬性
    private String product_id;
    private String category_id;
    private String product_name;
    private String product_desc;
    private int needmiles;
    private int quantity;
    private String product_image;

    //getter setter
    public String getProduct_id() {
        return product_id;
    }
    public void setProduct_id(String product_id) {
        this.product_id = product_id;
    }
    public String getCategory_id() {
```


首頁

view

接收使用者輸入文字，
再進到後端進行模糊查詢

連結到新增與查詢全部的頁面

```
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Redeem System</title>
<link rel="stylesheet" href="style.css">
</head>
<body>

<%@ include file="/topic2/navtest.html"%>
<div class="content">
<h2>贈品管理系統</h2>
    <form method="post" action="GetByName">
        輸入商品名稱 : <input type="text" name="product_name" /><p>
        <input type="submit" value="查詢" />
    </form>
    <a href="http://localhost:8080/FlightTicketingSystem/InsertProduct.jsp" class="li">
    <a href="http://localhost:8080/FlightTicketingSystem/GetAllProducts" class="li">
```

新增商品

view

新增對應資料傳至後端處理

```
1 <%@ page language="java" contentType="text/html; charset=UTF-8"
2   pageEncoding="UTF-8"%>
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6 <title>Insert Product</title>
7 <link rel="stylesheet" href="style.css">
8 </head>
9 <body>
10 <%@ include file="/topic2/navtest.html"%>
11 <div class="content">
12 <h2>新增資料</h2>
13 <form method="post" action="InsertProduct">
14 輸入類別編號:<input type="text" name="category_id" /><br>
15 輸入商品名稱:<input type="text" name="product_name" /><br>
16 輸入商品描述:<input type="text" name="product_desc" /><br>
17 兌換所需里程:<input type="text" name="needmiles" /><br>
18 輸入商品庫存:<input type="text" name="quantity" /><br>
19 商品圖片檔名:<input type="text" name="product_image" /><br>
20 <input type="submit" value="確定新增" />
21 </form>
22 備註：商品編號由系統自動產生<br>
23 </div>
24 </body>
25 </html>
```

修改商品

view

從GetAllProduct.jsp的修改按鈕跳過來

帶入該商品的product_id以及該商品資料，除了ID之外使用者都可以修改。

將資料傳至後端處理

```
<%= product.getNeedmiles() %>
<td>
<%= product.getQuantity() %>
<td>
<%= product.getProduct_image() %>
</td>
</table>
</div>
<form method="post" action="UpdateById">
    帶入商品編號：<input type="text" readonly name="product_id" placeholder="(系統帶入編號)" />
    輸入類別編號：<input type="text" name="category_id" value="<%= product.getCategory_id() %>" />
    輸入商品名稱：<input type="text" name="product_name" value="<%= product.getProduct_name() %>" />
    輸入商品描述：<input type="text" name="product_desc" value="<%= product.getProduct_desc() %>" />
    兌換所需里程：<input type="text" name="needmiles" value="<%= product.getNeedmiles() %>" />
    輸入商品庫存：<input type="text" name="quantity" value="<%= product.getQuantity() %>" />
    商品圖片檔名：<input type="text" name="product_image" value="<%= product.getProduct_image() %>" />
    <input type="submit" value="確定修改" />
</form>
```

查詢全部商品

View

request.getAttribute取得
request內資料

取得全部商品資料顯示在頁面上
新增、修改、刪除也會跳回這個頁面

每筆資料有修改和刪除按鈕
修改:按下去跳到修改商品的頁面
刪除:確認是否刪除

```
<table border="1">
  <tr>
    <th>編號</th><th>類別</th><th>名稱</th><th>描述</th><th>兌換所需里程</th><th>庫存</th><th>圖片檔案</th>
    <th>修改</th><th>刪除</th>
    <% List<ProductsBean> products=(ArrayList<ProductsBean>)request.getAttribute("products");
    for(ProductsBean product : products){ %>

  <tr>
    <td><%= product.getProduct_id() %>
    <td><%= product.getCategory_id() %>
    <td><%= product.getProduct_name() %>
    <td><%= product.getProduct_desc() %>
    <td><%= product.getNeedmiles() %>
    <td><%= product.getQuantity() %>
    <td><%= product.getProduct_image() %>
    <td>
      <form action="GetUpdate" method="post" >
        <input type="hidden" name="product_id" value="<%= product.getProduct_id() %>" />
        <button type="submit">修改</button>
      </form>
    <td>
      <form id="deleteform" action="DeleteProduct" method="post" >
        <input type="hidden" name="product_id" value="<%= product.getProduct_id() %>" />
        <button type="button" onclick="confirmDelete(this)">刪除</button>
      </form>
    </td>
  </tr>
</table>

</div><h3>共<%= products.size() %>筆商品資料</h3>
</div>
<script>
function confirmDelete(button){
  if(confirm('確定要刪除這項商品嗎?')){
    button.closest('form').submit();
  }
}
```


新增商品

Controller

將輸入值轉換成所需的格式

透過Dao內Insert方法新增至
Products資料表

若成功，則導向查全部的servlet
若失敗，則跳到失敗頁面

```
try {  
  
    // 從 HTML 獲取數據  
    String category_id = request.getParameter("category_id");  
    String product_name = request.getParameter("product_name");  
    String product_desc = request.getParameter("product_desc");  
    int needmiles = Integer.parseInt(request.getParameter("needmiles"));  
    int quantity = Integer.parseInt(request.getParameter("quantity"));  
    String product_image = request.getParameter("product_image");  
  
    //存進Bean  
    ProductsBean product = new ProductsBean();  
    product.setCategory_id(category_id);  
    product.setProduct_name(product_name);  
    product.setProduct_desc(product_desc);  
    product.setNeedmiles(needmiles);  
    product.setQuantity(quantity);  
    product.setProduct_image(product_image);  
  
    RedeemDao redeemDao = new RedeemDao();  
    boolean isSuccess = redeemDao.insertone(product);  
    if (isSuccess) {  
        //新增成功後，導向 查看全部商品的 Servlet  
        response.sendRedirect("GetAllProducts");  
    } else {  
        // 跳轉到結果頁面  
        request.getRequestDispatcher("Fail.jsp").forward(request, response);  
    }  
  
} catch (Exception e) {  
    e.printStackTrace();  
}
```

刪除商品 Controller

將輸入值轉換成所需的格式

透過Dao內delete方法刪除資料庫
資料

若成功，則導向查全部的servlet
若失敗，則跳到失敗頁面

```
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    try {
        // 從 HTML 獲取數據
        String product_id = request.getParameter("product_id");

        //調用delete方法
        RedeemDao redeemDao = new RedeemDao();
        boolean isSuccess = redeemDao.delete(product_id);

        if (isSuccess) {
            //刪除成功後，導向 查看全部商品的 Servlet
            response.sendRedirect("GetAllProducts");
        } else {
            // 跳轉到結果頁面
            request.getRequestDispatcher("Fail.jsp").forward(request, response);
        }
    }
}
```

修改商品

Controller

從 HTML 獲取商品id

透過Dao內getOneProduct方法拿到該商品資料

用request傳送bean去到
GetUpdate.jsp

```
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    try {
        // 從 HTML 獲取數據
        String product_id = request.getParameter("product_id");
        ProductsBean product = new ProductsBean();

        RedeemDao redeemDao = new RedeemDao();
        product=redeemDao.getOneProduct(product_id);

        request.setAttribute("product", product);
        request.getRequestDispatcher("GetUpdate.jsp").forward(request, response);

    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

修改商品 Controller

從 HTML 獲取修改的資料

資料存入Bean

調用dao的UpdateById方法

若成功，則導向查全部的servlet

若失敗，則跳到失敗頁面

```
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    try {

        // 從html獲得數據
        String product_id = request.getParameter("product_id");
        String category_id = request.getParameter("category_id");
        String product_name = request.getParameter("product_name");
        String product_desc = request.getParameter("product_desc");
        int needmiles = Integer.parseInt(request.getParameter("needmiles"));
        int quantity = Integer.parseInt(request.getParameter("quantity"));
        String product_image = request.getParameter("product_image");

        將數據存入 Bean
        ProductsBean product = new ProductsBean();
        product.setProduct_id(product_id);
        product.setCategory_id(category_id);
        product.setProduct_name(product_name);
        product.setProduct_desc(product_desc);
        product.setNeedmiles(needmiles);
        product.setQuantity(quantity);
        product.setProduct_image(product_image);

        RedeemDao redeemDao = new RedeemDao();
        ProductsBean success = redeemDao.UpdateById(product);

        if (success!=null) {
            response.sendRedirect("GetAllProducts");
        } else {
            request.getRequestDispatcher("Fail.jsp").forward(request, response)
        }

    }
```


查詢全部商品 controller

調用dao 的getall方法取得所有商品
資料放進List

用request傳遞資料給
GetAllProducts.jsp

如果成功就跳到
GetAllProducts.jsp，失敗就跳到
Fail.jsp

```
@SuppressWarnings("unused")
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    try {

        RedeemDao redeemDao = new RedeemDao();
        List<ProductsBean> products = redeemDao.getall();
        request.setAttribute("products", products);
        request.getRequestDispatcher("GetAllProducts.jsp").forward(request, response);

        if (products != null) {
            // 跳轉到成功頁面
            request.getRequestDispatcher("GetAllProducts.jsp").forward(request, response);
        } else {
            // 跳轉到失敗頁面
            request.getRequestDispatcher("Fail.jsp").forward(request, response);
        }
    } catch (Exception e) {
    }
}
```

根據名稱查詢 controller

調用dao 的getByName方法取得多筆商品資料放進List

用request傳遞資料給
GetAllProducts.jsp

如果成功就跳到
GetAllProducts.jsp，失敗就跳到
Fail.jsp

```
@WebServlet("/GetByName")
public class GetByName extends HttpServlet {
    private static final long serialVersionUID = 1L;

    //用商品名稱查詢多筆商品資料
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        try {
            String product_name = request.getParameter("product_name");
            RedeemDao redeemDao = new RedeemDao();
            List<ProductsBean> products = redeemDao.getByName(product_name);
            request.setAttribute("products", products);
            request.getRequestDispatcher("GetAllProducts.jsp").forward(request, response);
        } catch (ServletException | IOException e) {
            e.printStackTrace();
        }
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        doGet(request, response);
    }
}
```

Dao

- 用id查詢一筆商品資料
- 用名稱模糊查詢多筆商品資料
- 查詢全部商品
- 新增一筆商品資料
- 刪除一筆商品資料by Id
- 修改一筆商品資料by Id

```
String sql = "INSERT INTO Products (category_id, product_name, product_desc, ne
int generatedId = -1;
try {
    // 新增產品資料

    conn = Utils.getConnection();

    stmt = conn.prepareStatement(sql, PreparedStatement.RETURN_GENERATED_KEYS);
    stmt.setString(1, product.getCategory_id());
    stmt.setString(2, product.getProduct_name());
    stmt.setString(3, product.getProduct_desc());
    stmt.setInt(4, product.getNeedmiles());
    stmt.setInt(5, product.getQuantity());
    stmt.setString(6, product.getProduct_image());

    // 執行 INSERT
    int affectedRows = stmt.executeUpdate();

    // 確保有新增資料
    if (affectedRows > 0) {
        // 取得自動生成的 product_id
        try (ResultSet generatedKeys = stmt.getGeneratedKeys()) {
            if (generatedKeys.next()) {
                generatedId = generatedKeys.getInt(1);
            }
        }

        return affectedRows > 0;
    }
}
```

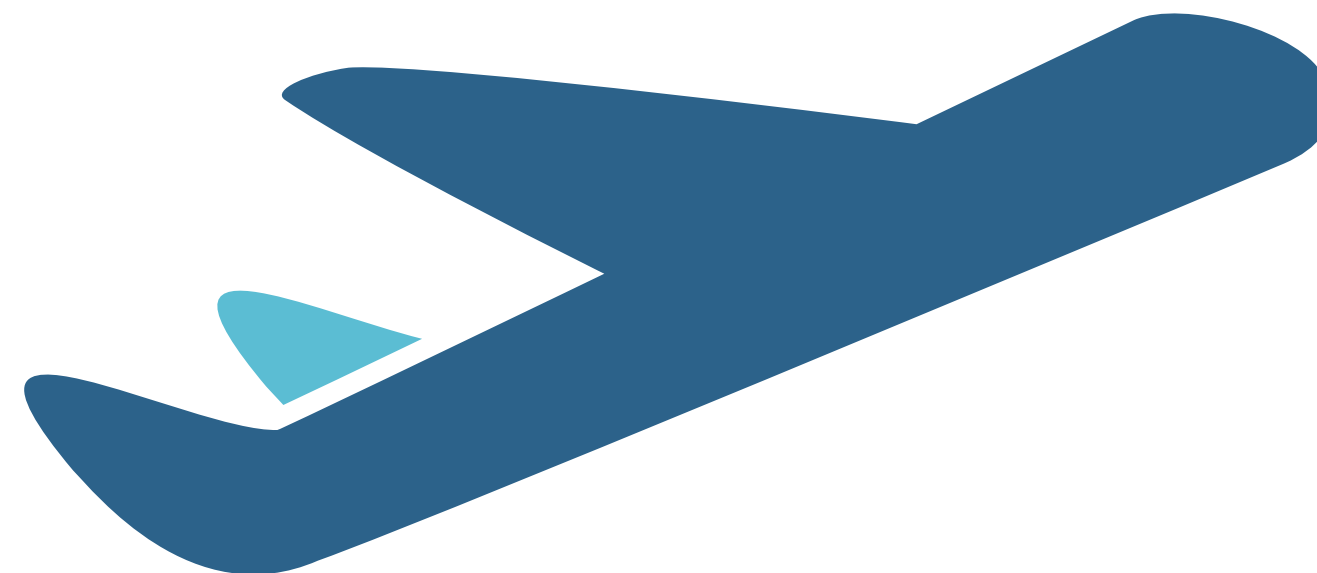


機場管理

1. 機場管理

- 新增 - 修改 - 刪除 - 查詢

負責組員：蔡岳峰



Airports

表格名稱	中文：機場資料庫 英文：Airports		索引鍵：		
主鍵：	member_id		外來鍵：		
欄位編號	欄位名稱	欄位敘述	資料型態	欄位長度	備註
1	AirportsId	機場ID	INT		PK(自增)
2	IataCode	機場IATA代碼	char	3	NOT NULL
3	AirportName	機場名稱	NVARCHAR	100	NOT NULL
4	City	城市	NVARCHAR	100	NOT NULL
5	CountryRegion	國家	NVARCHAR	100	NOT NULL

機場管理MVC



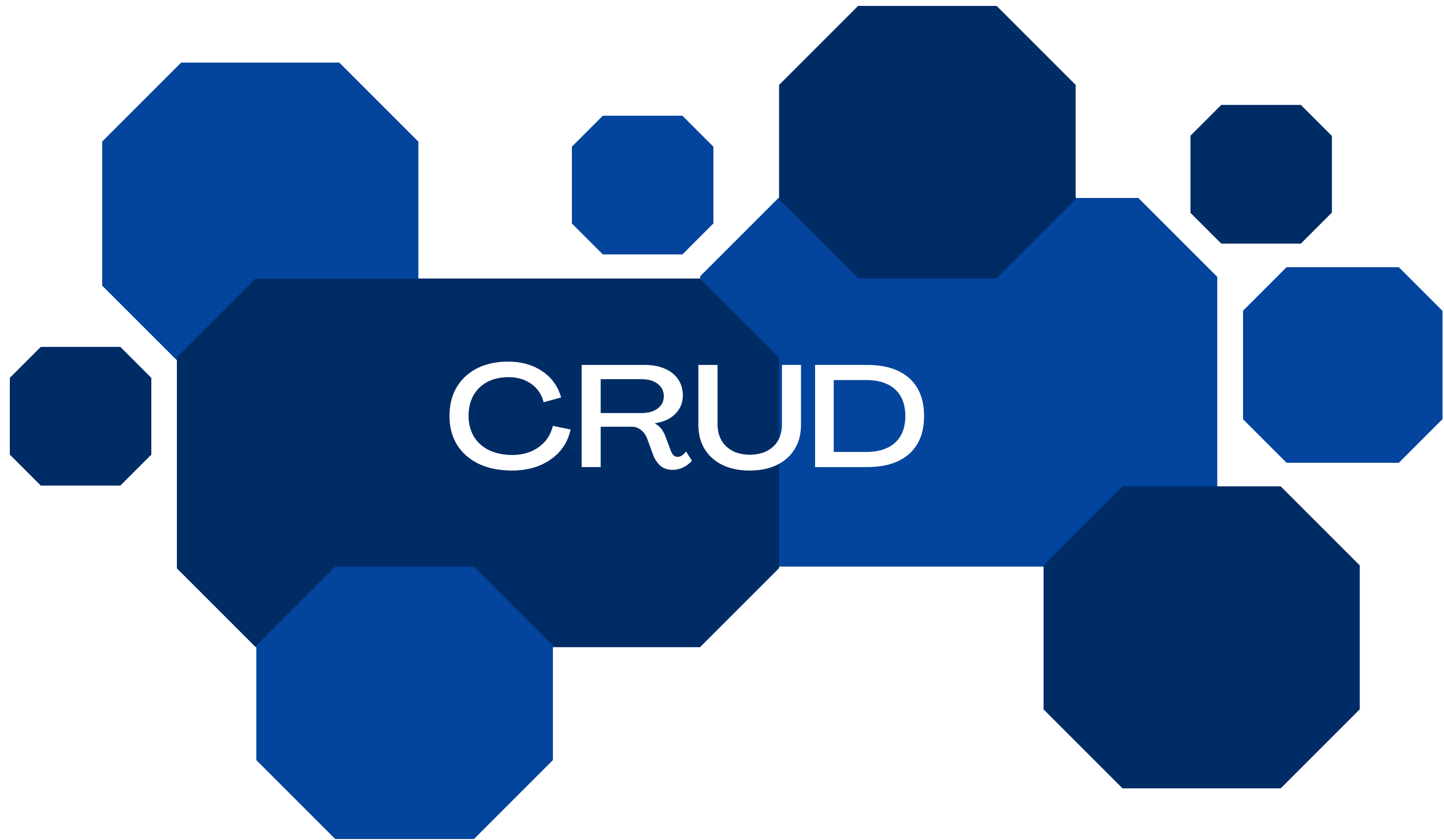


// 編輯機場函數

```
function updateAirport() {  
  const form = document.getElementById("airportForm");  
  const updatedAirportPR = {  
    AirportsId: form.airportsId.value,  
    IataCode: form.iataCode.value,  
    AirportName: form.airportName.value,  
    City: form.city.value,  
    CountryRegion: form.country.value,  
  };  
  axios.post("../airports", { action: "update", ...updatedAirportPR })  
    .then(response => {  
      if (response.data.status) {  
        searchAirports(); // 重新獲取機場資料  
        document.getElementById("airportDialog").close(); // 關閉對話框  
      } else {  
        alert("編輯機場失敗: " + response.data.message);  
      }  
    })  
    .catch(error => console.error("編輯機場失敗:", error));  
}
```

Bean

```
2
3 public class AirportsBean {
4
5     private int AirportsId; // 機場 ID
6     private String IataCode; // 機場代碼
7     private String AirportName; // 機場名稱
8     private String CountryRegion; // 機場所處國家
9     private String City; // 機場所處城市
10
11 }
```



新增

```
//新增機場
public void insertAirport(AirportsBean airport) throws SQLException {
    String sql = "INSERT INTO Airports (AirportName, IataCode, CountryRegion,City) VALUES (?, ?, ?,?)";
    Connection connection = JDBCUtil.getConnection();
    PreparedStatement preparedStatement = null;

    try {
        preparedStatement = connection.prepareStatement(sql);
        preparedStatement.setString(1, airport.getAirportName());
        preparedStatement.setString(2, airport.getIataCode());
        preparedStatement.setString(3, airport.getCountryRegion());
        preparedStatement.setString(4, airport.getCity());
        preparedStatement.executeUpdate(); // 執行更新
    } catch (SQLException e) {
        e.printStackTrace();
        throw e;
    } finally {
        JDBCUtil.closeResources(connection, preparedStatement); // 確保關閉所有資源
    }
}
```


刪除

```
21
//刪除機場
public void deleteAirport(int airportsId) throws SQLException {
    String sql = "DELETE FROM Airports WHERE AirportsId = ?";
    Connection connection = JDBCUtil.getConnection();
    PreparedStatement preparedStatement = null;

    try {
        preparedStatement = connection.prepareStatement(sql);
        preparedStatement.setInt(1, airportsId);
        preparedStatement.executeUpdate(); // 執行更新
    } catch (SQLException e) {
        e.printStackTrace();
        throw e;
    } finally {
        JDBCUtil.closeResources(connection, preparedStatement, null); // 確保關閉所有資源
    }
}
```

修改

```
// 修改機場
public void updateAirports(List<AirportsBean> airports) throws SQLException {
    String sql = "UPDATE Airports SET AirportName = ?, IataCode = ?, CountryRegion = ?, City = ? WHERE AirportsId = ?";
    Connection connection = JDBCUtil.getConnection();
    PreparedStatement preparedStatement = null;

    try {
        preparedStatement = connection.prepareStatement(sql);

        for (AirportsBean airport : airports) {
            preparedStatement.setString(1, airport.getAirportName());
            preparedStatement.setString(2, airport.getIataCode());
            preparedStatement.setString(3, airport.getCountryRegion());
            preparedStatement.setString(4, airport.getCity());
            preparedStatement.setInt(5, airport.getAirportsId()); // 設置要更新的機場 ID
            preparedStatement.addBatch(); // 將更新添加到批次中
        }

        preparedStatement.executeBatch(); // 執行批次更新
    } catch (SQLException e) {
        e.printStackTrace();
        throw e;
    } finally {
        JDBCUtil.closeResources(connection, preparedStatement, null); // 確保關閉所有資源
    }
}
```

查詢

```
//搜尋機場
public ArrayList<AirportsBean> searchAirports(String keyword, String city, String countryRegion) throws SQLException {

    ArrayList<AirportsBean> airportsList = new ArrayList<>();
    StringBuilder sql = new StringBuilder("SELECT * FROM Airports WHERE 1=1"); // 基本查詢
    Connection connection = JDBCUtil.getConnection();
    ResultSet resultSet = null;
    PreparedStatement preparedStatement = null;
    // 用於存儲查詢參數的列表
    List<String> parameters = new ArrayList<>();

    // 根據條件動態添加查詢
    if (keyword != null && !keyword.trim().isEmpty()) {
        sql.append(" AND AirportName LIKE ? OR IataCode LIKE ?"); // 使用 OR 來搜尋
        parameters.add("%" + keyword + "%");
        parameters.add("%" + keyword + "%");
    }
    if (city != null && !city.trim().isEmpty() && !city.equals("全部")) {
        sql.append(" AND City = ?");
        parameters.add(city); // 不需要加 '%'，因為這是精確匹配
    }
    if (countryRegion != null && !countryRegion.trim().isEmpty()) {
        sql.append(" AND CountryRegion LIKE ?");
        parameters.add("%" + countryRegion + "%");
    }

    try {
        preparedStatement = connection.prepareStatement(sql.toString());

        for (int i = 0; i < parameters.size(); i++) {
            preparedStatement.setString(i + 1, parameters.get(i));
        }
        // 打印生成的 SQL 語句和參數

        resultSet = preparedStatement.executeQuery();
        while (resultSet.next()) {
            AirportsBean airport = new AirportsBean();
            airport.setAirportsId(resultSet.getInt("AirportsId"));
            airport.setAirportName(resultSet.getString("AirportName"));
            airport.setIataCode(resultSet.getString("IataCode"));
            airport.setCountryRegion(resultSet.getString("CountryRegion"));
            airport.setCity(resultSet.getString("City"));
            airportsList.add(airport); // 將每個案例添加到列表中
        }
    } catch (SQLException e) {

        throw e;
    }
    finally {
        JDBCUtil.closeResources(connection, preparedStatement, resultSet); // 確保關閉所有資源
    }

    return airportsList; // 返回包含搜尋結果的列表
}
```

搜尋全部

```
//搜尋全部
public ArrayList<AirportsBean> SELECTallinfo() throws SQLException{
    String sql = "SELECT * FROM Airports";
    Connection connection = JDBCUtil.getConnection();
    PreparedStatement preparedStatement = null;
    ResultSet resultSet = null;
    ArrayList<AirportsBean> AirportsList = new ArrayList<>();
    try {
        preparedStatement = connection.prepareStatement(sql);
        resultSet = preparedStatement.executeQuery(); // 使用 executeQuery() 來獲取結果集
        while (resultSet.next()) {

            AirportsBean Airports = new AirportsBean();
            Airports.setAirportsId(resultSet.getInt("AirportsId"));
            Airports.setAirportName(resultSet.getString("AirportName"));
            Airports.setIataCode(resultSet.getString("IataCode"));
            Airports.setCountryRegion(resultSet.getString("CountryRegion"));
            Airports.setCity(resultSet.getString("City"));

            AirportsList.add(Airports); // 將每個案例添加到列表中
        }
    } catch (SQLException e) {
        e.printStackTrace();
        throw e;
    } finally {
        JDBCUtil.closeResources(connection, preparedStatement, resultSet); // 確保關閉所有資源
    }
    return AirportsList; // 返回包含所有案例的列表
}
```

SERVLET

```
0
1 @WebServlet("/airports")
2 public class airports extends HttpServlet {
3     private static final long serialVersionUID = 1L;
4     Map<String, Object> jsonResponse = new HashMap<>();
5     boolean status = true;
6
7     public airports() {
8         super();
9     }
10
11     protected void doPost(HttpServletRequest request, HttpServletResponse response)
12         throws ServletException, IOException {
13         // 設定回應內容類型
14         response.setContentType("application/json");
15         jsonResponse.clear();
16         // 使用自定義函式解析 JSON 數據
17         Map<String, String> params = parseJsonRequest(request);
18
19         // 獲取操作類型
20         String action = params.get("action");
21         AirportsDao aDao = new AirportsDao();
22         ArrayList<AirportsBean> AirportsList = null;
```



```

try {

    //取的所有機場
    if (action.equalsIgnoreCase("getall")) {
        AirportsList = aDao.SELECTallinfo();
        jsonResponse.put("result", AirportsList);

    } else if (action.equalsIgnoreCase("search")) {

        String city=params.get("city");|
        String country=params.get("country");
        String keyword=params.get("keyword");

        AirportsList=aDao.searchAirports(keyword,city,country);
        jsonResponse.put("result", AirportsList);

        //更新機場資料
    } else if (action.equalsIgnoreCase("update")) {

        AirportsBean newAirport = new AirportsBean();
        ArrayList<AirportsBean> updateAirports = new ArrayList<>();
        newAirport.setAirportsId(Integer.parseInt(params.get("AirportsId")));
        newAirport.setIataCode(params.get("IataCode"));
        newAirport.setAirportName(params.get("AirportName"));
        newAirport.setCity(params.get("City"));
        newAirport.setCountryRegion(params.get("CountryRegion"));
        updateAirports.add(newAirport);
        aDao.updateAirports(updateAirports);
        //新增機場資料
    } else if (action.equalsIgnoreCase("insert")) {

        AirportsBean newAirport = new AirportsBean();

        newAirport.setIataCode(params.get("IataCode"));
        newAirport.setAirportName(params.get("AirportName"));
        newAirport.setCity(params.get("City"));
        newAirport.setCountryRegion(params.get("CountryRegion"));

        aDao.insertAirport(newAirport);
        //刪除機場資料
    } else if (action.equalsIgnoreCase("delete")) {

        int delectid = Integer.parseInt(request.getParameter("airportsId"));
        aDao.deleteAirport(delectid);

    }
} catch (Exception e) {
    e.printStackTrace();
    response.setStatus(HttpServletResponse.SC_INTERNAL_SERVER_ERROR); // 設置 HTTP 錯誤狀態碼

    return;
}

```

```
jsonResponse.put("status", status);  
// 使用 Gson 將列表轉換為 JSON  
Gson gson = new Gson();  
String json = gson.toJson(jsonResponse);  
  
// 設置回應類型為 JSON  
response.setContentType("application/json");  
response.setCharacterEncoding("UTF-8");  
response.getWriter().write(json); // 寫入 JSON 到回應
```



```
1 // 初始取得全部機場資料供搜尋
```

```
2     const allairport = [];
```

```
3     axios
```

```
4         .post("../airports", { action: "getall" })
```

```
5         .then((response) => {
```

```
6             allairport.push(...response.data.result);
```

```
7         })
```

```
8         .catch((error) => console.error("獲取機場資料失敗:", error));
```

前端頁面

城市:

請選擇城市

國家:

請選擇國家

請輸入關鍵字

🔍 搜尋機場

+ 新增機場

機場 ID	機場IATA代碼	機場名稱	城市	國家	操作
1	TSA	松山機場	台北市	台灣	編輯 刪除
2	AAA	阿納機場	阿納	法屬玻里尼西亞	編輯 刪除
3	AAB	阿拉貝里機場	阿拉貝里	澳洲	編輯 刪除
4	AAC	阿里什國際機場	阿里什	埃及	編輯 刪除
5	AAE	拉巴赫·比塔特機場	拉巴赫	阿爾及利亞	編輯 刪除
6	AAF	阿巴拉契科拉支線機場	阿巴拉契科拉	美國	編輯 刪除
7	AAG	阿拉波蒂機場	阿拉波蒂	巴西	編輯 刪除

城市:

請選擇城市

全部

台北市

阿納

阿拉貝里

阿里什

拉巴赫

國家:

請選擇國家

全部

台灣

法屬玻里尼西亞

澳洲

埃及

阿爾及利亞



1 城市:

```
2     <div class="drop">
3         <div class="dropOption">
4             <input type="search" class="searchcity" id="searchcity" name="searchcity" placeholder="請選擇城市"/>
5         </div>
6         <ul class="dropdown" id="dropdowncity"></ul>
7     </div>
```



```
1 // 尋找匹配的資料
2 function findMatches(wordToMatch, airporties, type) {
3     const regex = new RegExp(wordToMatch, "gi"); // 建立正則表達式
4     const matches = airporties.filter((place) => place[type].match(regex)); // 根據指定類型進行匹配
5
6     // 去除重複項目
7     const uniqueMatches = Array.from(new Set(matches.map(place => place[type]))).map(name => {
8         return matches.find(place => place[type] === name);
9     });
10
11     return uniqueMatches;
12 }
```



```
1 // 根據類型選擇正確的下拉選單
2     if (id === "searchcity") {
3
4         suggestionscity.innerHTML = html;
5     } else if (id === "searchcountry") {
6
7         suggestionscountry.innerHTML = html;
8     } else if (id === "searchinputcountry") {
9
10        suggestionsinputcountry.innerHTML = html;
11    }
12    else if (id === "searchinputcity") {
13
14        suggestioninputcity.innerHTML = html;
15    }
```



```
1  // 根據類型選擇正確的下拉選單
2      if (id === "searchcity") {
3
4          suggestionscity.innerHTML = html;
5      } else if (id === "searchcountry") {
6
7          suggestionscountry.innerHTML = html;
8      } else if (id === "searchinputcountry") {
9
10         suggestionsinputcountry.innerHTML = html;
11     }
12     else if (id === "searchinputcity") {
13
14         suggestioninputcity.innerHTML = html;
15     }
```

新增

機場IATA代碼:

機場名稱:

城市:

國家:

確定

取消

修改

機場 ID:

機場IATA代碼:

機場名稱:

城市:

國家:

確定

取消

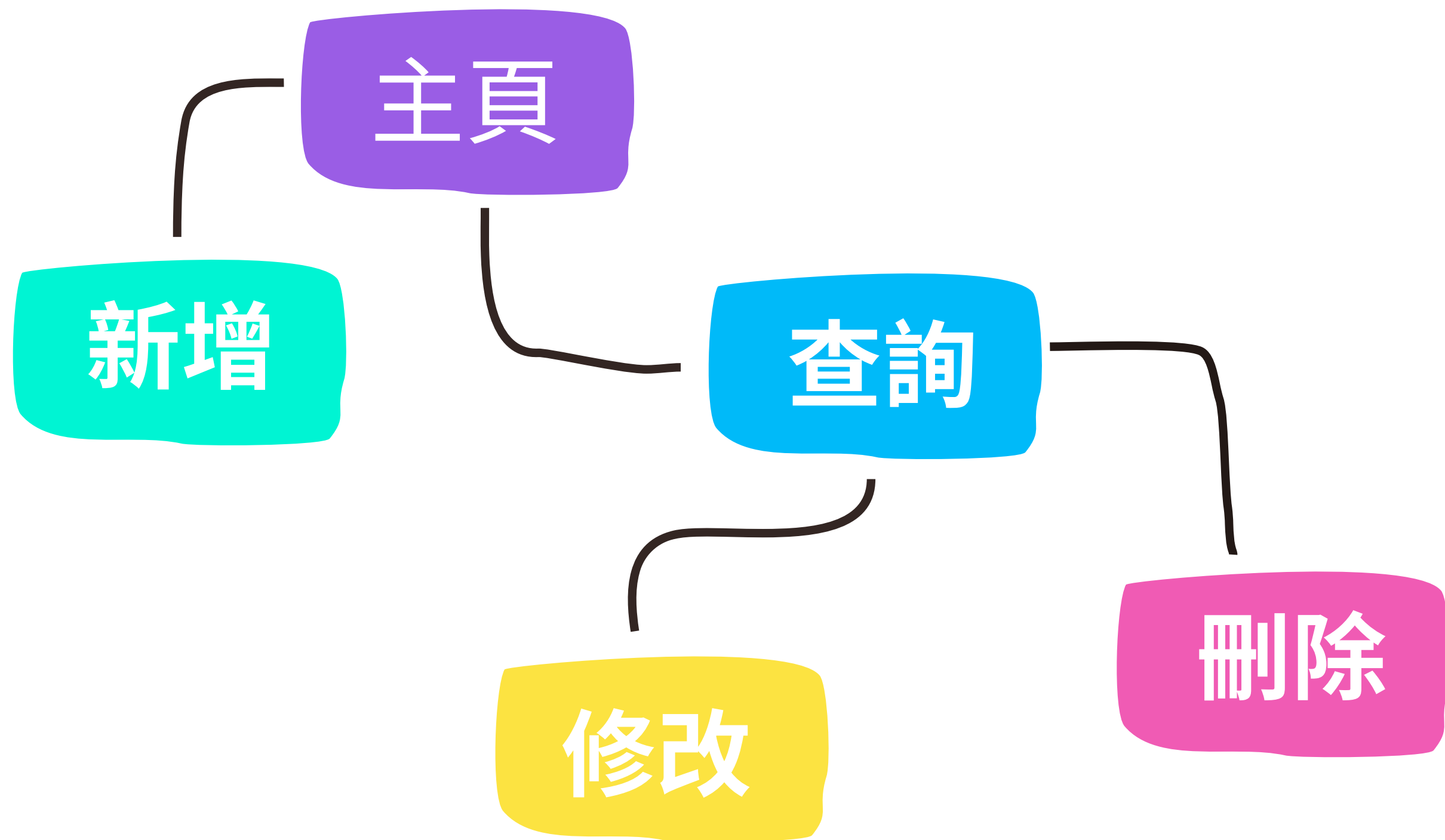


航班管理

1

負責組員：郭俊宏

▶ 流程圖



主要功能

前端功能

- 動態畫面渲染
- LOGO超鏈接回主頁
- 機場交換按鍵
- 提示彈窗
- 資料全空檢查
- 搜索欄AUTOCOMplete

后端功能

- **CRUD**航班資料
- 資料驗證
- 機場名或IATA代碼查詢API提供
AUTOCOMplete

檔案架構

```
SRC/
|--- MAIN/
|   |--- JAVA/                                # JAVA原始碼
|   |   |--- COM/ GITHUB/JIUNHUNGGUO          # 公司或組織域名（反向）
|   |   |   |--- FLIGHTBOOKING/              # 專案名稱
|   |   |   |   |--- FLIGHTMANAGEMENT/        # 模組或應用名稱
|   |   |   |       |--- CONFIG/              # 配置類（安全配置）
|   |   |   |       |--- CONTROLLER/          # 控制器層（處理HTTP請求）
|   |   |   |       |--- SERVICE/            # 服務層（業務邏輯）
|   |   |   |       |--- REPOSITORY/          # 資料存取層（DAO）
|   |   |   |       |--- MODEL/              # 實體類（航班資料實體）
|   |   |   |       |--- UTIL/              # 工具類
|   |   |   |       |--- MAIN方法檔案
|   |--- RESOURCES/                          # 資源文件
|   |   |--- STATIC/                        # 靜態資源（CSS、JS、圖片等）
|   |   |--- TEMPLATES/                    # 模板文件（如THYMELEAF、FREEMARKER）
|   |   |--- APPLICATION.PROPERTIES        # 主配置文件
|--- WEBAPP/                                # WEB資源（可選，用於傳統WEB專案）
|--- POM.XML                                # MAVEN配置文件
```


Flights

	Column Name	Data Type	Allow Nulls
	id	int	☐
	flightNumber	nvarchar(50)	☐
	aircraftCode	nvarchar(50)	☒
	airlineName	nvarchar(50)	☐
	originIata	nvarchar(50)	☐
	originName	nvarchar(50)	☐
	destinationIata	nvarchar(50)	☐
	destinationName	nvarchar(50)	☐
	departureTime	int	☒
	arrivalTime	int	☒
			☐

資料來源



[Home](#) [Pricing](#) [Contact](#) [Login](#)

Getting Started!

-  [Oneway Trip API](#)
-  [Round Trip API](#)
-  [Multi Trip API](#)
-  [Flight Tracking API](#)
-  [Track Flights between Airports](#)
-  [Airline & Airport code API](#)
-  [Airport Schedule API](#)



Airport Schedule API

This API will provide you with complete schedule of any airport. You can get data according to arrivals and departures separately.

API endpoint for this API is: `https://api.flightapi.io/schedule`

Guide

Your API requests are authenticated using API keys. Any request that doesn't include an API key will return an error.

請求API

```
public class FlightApi {

    public static void fetchApiData(String apiUrl, String fileName, int totalPages) {
        StringBuilder mergedContent = new StringBuilder();
        mergedContent.append("{\"data\":[");

        for (int page = 1; page <= totalPages; page++) {
            String pagedUrl = apiUrl + "&page=" + page;

            try {
                URL url = new URL(pagedUrl);
                HttpURLConnection conn = (HttpURLConnection) url.openConnection();
                conn.setRequestMethod("GET");

                try (BufferedReader in = new BufferedReader(new InputStreamReader(conn.getInputStream()))) {

                    String inputLine;
                    while ((inputLine = in.readLine()) != null) {
                        mergedContent.append(inputLine);
                    }

                    if (page < totalPages) {
                        mergedContent.append(",");
                    }

                    System.out.println("Fetched page " + page + " of " + fileName);

                } catch (IOException e) {
                    System.err.println("Failed to read API response: " + e.getMessage());
                }

            } catch (Exception e) {
                System.err.println("API request failed: " + e.getMessage());
            }
        }

        mergedContent.append("]}");
    }
}
```

